
Bölüm-4

NESNEYE DAYALI PROGRAMLAMA “KALITIM”

OBJECTIVES



Bu bölümde;

- ▶ Varolan bir sınıfa dayalı olarak yeni sınıflar yaratmayı ve katılımı anlayacaksınız.
- ▶ Superclass ve subclass kavramlarını öğreneceksiniz.
- ▶ Bir sınıfın davranışlarını diğer sınıfa miras bırakmak için “extends” anahtar kelimesini öğreneceksiniz.
- ▶ “protected” erişim denetleyicisi ile superclass üyelerini nasıl kullandığınızı öğreneceksiniz.
- ▶ Superclass üyelerine “super” anahtar kelimesi ile erişeceksiniz.
- ▶ Kalıtım hiyerarşisinde constructor kullanımını öğreneceksiniz.
- ▶ Object sınıfının metotlarına hangi yollar ile erişildiğini öğreneceksiniz.



- ▶ Giriş
- ▶ Superclass ve subclass tanımlama
- ▶ “protected” üyeler
- ▶ Superclass vs subclass ilişkisi
- ▶ Subclass içinde constructor tanımlama
- ▶ OBJECT sınıfı
- ▶ Composition vs Inheritance
- ▶ Özet



► Kalıtım;

- Mevcut bir sınıfın üyelerini kullanarak, onları yeni / değiştirilmiş yeteneklerle süsleyerek yeni bir sınıf oluşturulur.
- Mevcut kanıtlanmış ve hatası ayıklanmış yüksek kaliteli yazılımlara dayalı yeni sınıflara dayalı program geliştirmek zamandan tasarruf sağlar.
- Bir sistemin etkin bir şekilde uygulanması ve sürdürülmesi olasılığını artırır.
- “has-a” ve “is-a” arasındaki ayrımı yapmak gerekir.
- Is-a kalıtımı temsil eder.
 - Bir is-a ilişkisinde, bir alt sınıfın nesnesi, üst sınıfının bir nesnesi olarak da ele alınabilir.
- Has-a kompozisyonu temsil eder.
 - Has-a ilişkisinde, bir nesne üye olarak diğer nesnelere referanslar içerir.



- ▶ Üst sınıflar "daha genel" iken, alt sınıflar "daha spesifik" olma eğilimindedir.
- ▶ Her alt sınıf nesnesi kendi üst sınıfının bir nesnesi olduğundan ve bir üst sınıfın birçok alt sınıfı olabilir.
- ▶ Bir üst sınıf tarafından temsil edilen nesneler kümesi, alt sınıflarından herhangi biri tarafından temsil edilen nesneler kümesinden tipik olarak daha büyüktür.

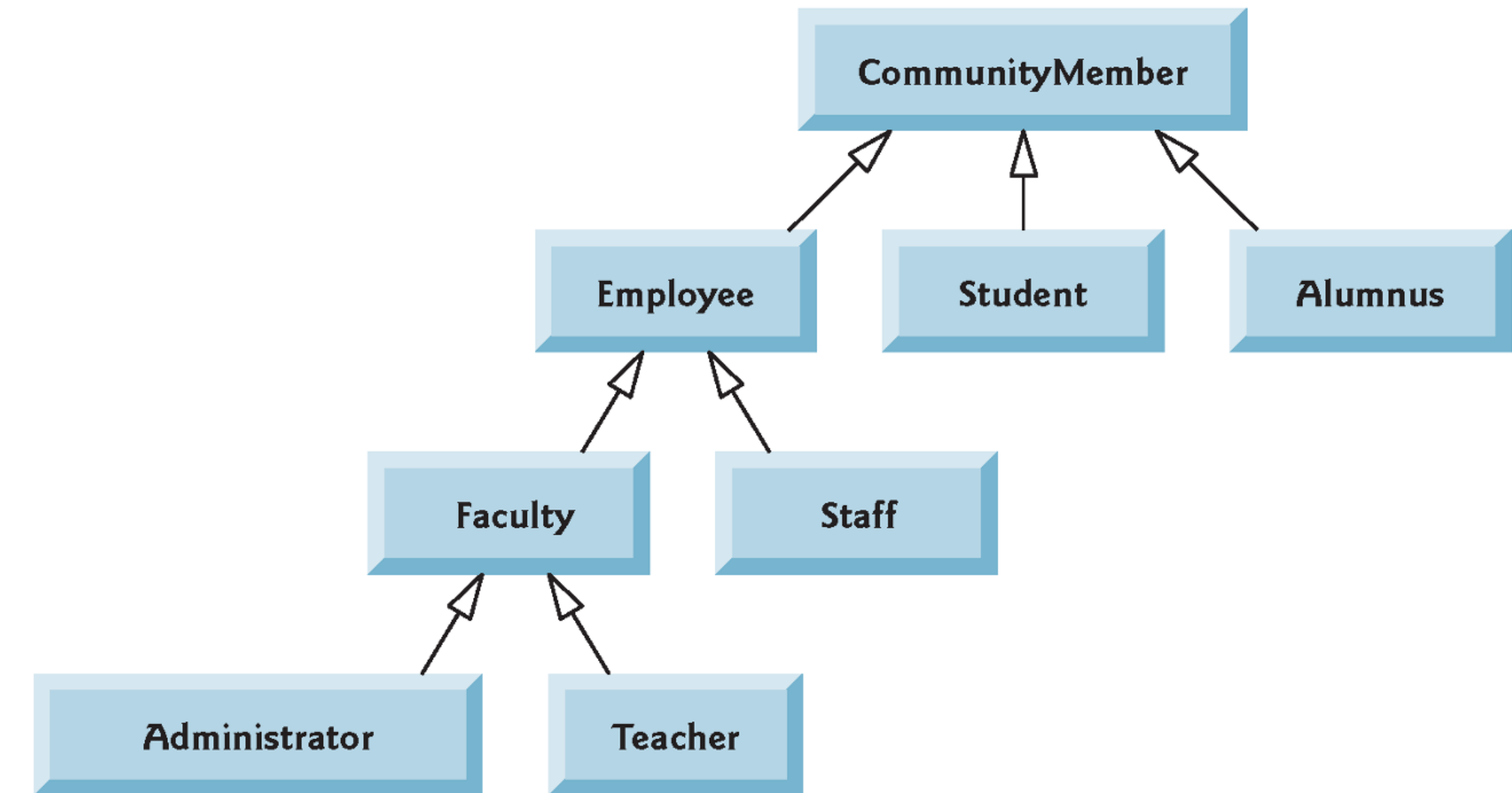
Superclass	Subclasses
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff
BankAccount	CheckingAccount, SavingsAccount

Fig. 9.1 | Inheritance examples.

SUPER VS SUB CLASS



- Bir üst sınıf, alt sınıflarıyla hiyerarşik bir ilişki (Kalıtım hiyerarşisi) içinde bulunur.
- Hiyerarşideki her ok bir “is-a” ilişkisini temsil eder.
- Sınıf hiyerarşisinde yukarı doğru oklar takip edilir.
- CommunityMember sınıfı
 - Employee, Student ve Alumnus için doğrudan,
 - Diyagramdaki diğer tüm sınıflar için dolaylı bir üst sınıfıdır.



| Inheritance hierarchy UML class diagram for university CommunityMembers.



- Yine birkaç “is-a” ilişkisini belirlemek için diyagramın altından bu sınıf hiyerarşisindeki en üst sınıfa kadar olan okları takip edebilirsiniz.
- Üçgen; İki Boyutlu Şekildir ve aynı zamanda bir Şekildir
- Küre; bir Üç Boyutlu Şekildir ve bir Şekildir.

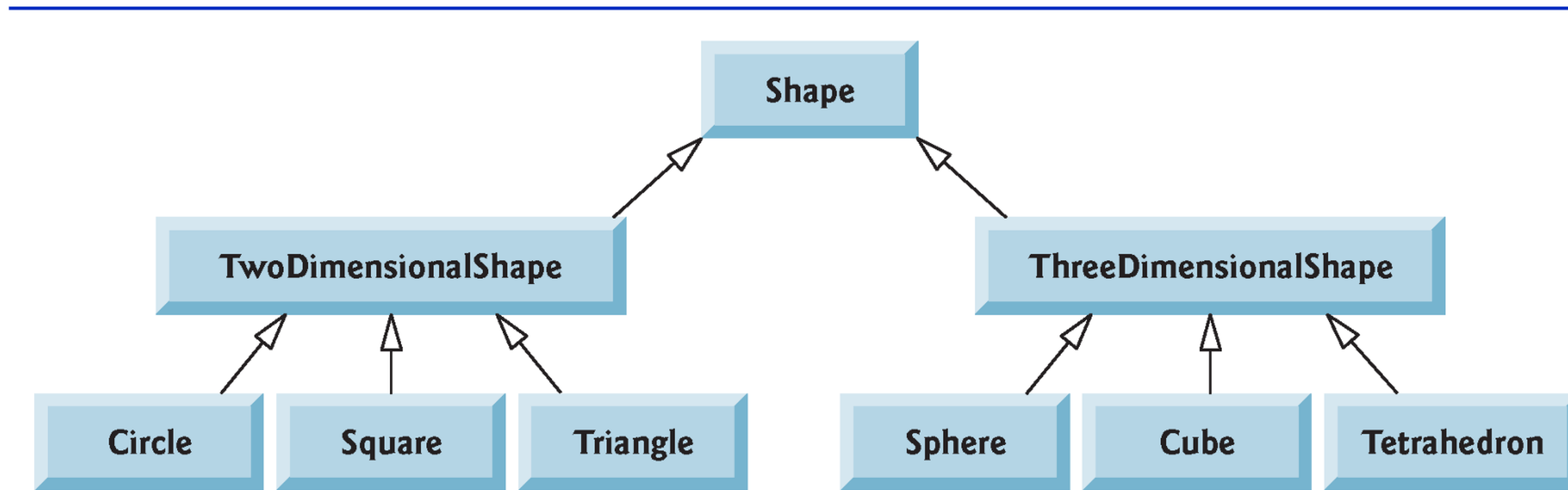


Fig. 9.3 | Inheritance hierarchy UML class diagram for Shapes.



- ▶ Her sınıf ilişkisi bir kalıtım ilişkisi olmak zorunda değildir.
- ▶ “is-a” ilişkisine ek olarak “has-a” ilişkisi de vardır.
- ▶ Mevcut sınıfların bileşimine göre sınıflar oluşturabilir;
 - ▶ Örnek: **Employee**, **BirthDate** ve **TelephoneNumber** sınıfları göz önüne alındığında;
 - ▶ Bir **Employee** bir **BirthDate** değildir. (An Employee is NOT a BirthDate.)
 - ▶ Bir **Employee** bir **TelephoneNumber** değildir.
- ▶ Ancak, her Employee için bir BirthDate ve TelephoneNumber vardır. (An Employee has a BirthDate.)

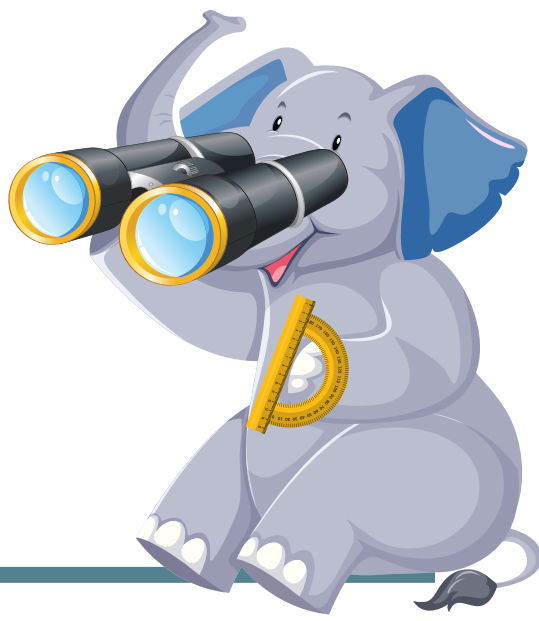


- ▶ Ortak bir üst sınıfı genişleten tüm sınıfların nesneleri, o üst sınıfın nesneleri olarak ele alınabilir.
- ▶ Miras sorunu;
 - ▶ Bir alt sınıf, ihtiyaç duymadığı veya sahip olmaması gereken yöntemleri de miras alabilir.
 - ▶ Bir alt sınıf için bir üst sınıf yöntemi uygun olsa bile, bu alt sınıf genellikle yöntemin özelleştirilmiş bir versiyonuna ihtiyaç duyar.
 - ▶ Alt sınıf, uygun bir uygulama ile üst sınıf yöntemini geçersiz kılabilir ve yeniden tanımlar (OVERRIDE).



- ▶ Bir sınıfın **public** üyelerine, programın o sınıfın bir nesnesine veya alt sınıflarından birine referansı olduğu her yerde erişilebilir.
- ▶ Bir sınıfın **private** üyelerine yalnızca sınıfın içinden erişilebilir.
- ▶ “**protected**” erişim, public ve private arasında bir orta düzey erişimdir.
 - ▶ Bir üst sınıfın **protected** üyelerine,
 - ▶ o üst sınıfın üyeleri,
 - ▶ alt sınıflarının üyeleri,
 - ▶ aynı paketteki diğer sınıfların üyeleri tarafından erişilebilir.
- ▶ **protected** üyelerin ayrıca paket erişimi (package access) vardır.
- ▶ Tüm public ve **protected** üst sınıf üyeleri, alt sınıfın üyesi olduklarında orijinal erişim değiştiricilerini korur.

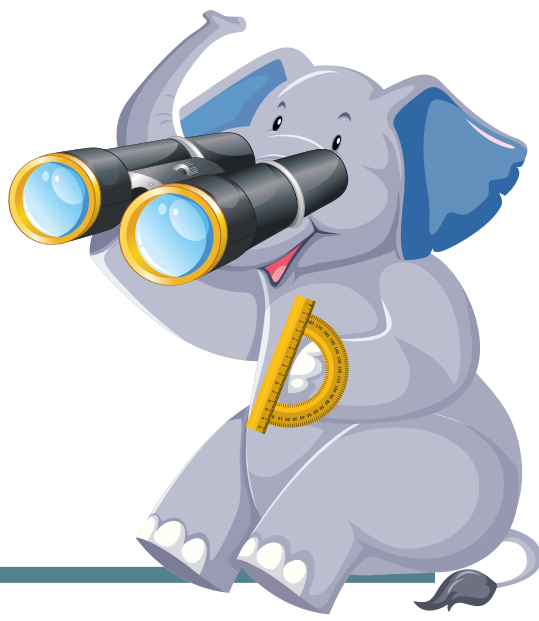
SOFTWARE ENGINEERING OBSERVATION



Bir alt sınıfın metotları, üst sınıflarının private üyelerine doğrudan erişemez.

Bir alt sınıf, private üst sınıf örnek değişkenlerinin durumunu yalnızca üst sınıfta sağlanan ve alt sınıf tarafından devralınan private olmayan metotlarla değiştirebilir.

SOFTWARE ENGINEERING OBSERVATION



Private örnek değişkenlerini kullanmak, sistemleri test etmenize, hatalarını ayıklamanıza ve doğru şekilde değiştirmenize yardımcı olur.

Bir alt sınıf, kendi üst sınıfının private örnek değişkenlerine erişebilseydi; o alt sınıftan miras alan sınıflar da örnek değişkenlerine erişebilirdi.

Bu durum, private örnek değişkenleri olması gerekenlere erişimi yayar ve bilgi gizlemenin faydaları kaybolur.



- ▶ Bir şirketin bordro uygulamasında çalışan türleri içeren miras hiyerarşisi örneği.
- ▶ **Komisyon çalışanlarına** satışlarının bir yüzdesi ödenir. (Prim gibi düşünün.)
- ▶ **Baz maaşlı komisyon çalışanları**, baz maaşlarına ek olarak satışlarının bir yüzdesini alır.
- ▶ **CommissionEmployee**
 - ▶ extends Object
- ▶ **BasePlusCommissionEmployee**
 - ▶ extends Object
 - ▶ Extends CommissionEmployee





► Bu sınıflar arasındaki ilişki hakkındaki tartışmamızı beş örneğe ayırıyoruz.

1. `CommissionEmployee` (extends with `Object`)

2. `BasePlusCommissionEmployee` (extends with `Object`)

3. `BasePlusCommissionEmployee` (extends with `CommissionEmployee`)

► `Private` üyelere erişim denemesi.

4. `BasePlusCommissionEmployee` (extends with `CommissionEmployee`)

► `CommissionEmployee` sınıfı değişkenleri `protected` yapmak.

5. `BasePlusCommissionEmployee` (extends with `CommissionEmployee`)

► `CommissionEmployee` sınıfı değişkenleri `private` yapmak ve `Set-get` metotları kullanımı

► `super` ile üst metotlara erişmek



1. CommissionEmployee:

- ▶ Doğrudan Object sınıfından miras alan.
- ▶ Bir ad, soyadı, sosyal güvenlik numarası, komisyon oranı ve brüt (yani toplam) satış tutarı gibi özel örnek değişkenlere sahip olan.
- ▶ Set-get metotları bulunan.
- ▶ Kazancı yaptığı toplam satıştan aldığı komisyon ile hesaplanan.

Bir sınıf olarak kodlanır.

```
CommissionEmployee
CommissionEmployee(String, String, String, double, double)
grossSales double
lastName String
firstName String
socialSecurityNumber String
commissionRate double
getFirstName() String
setCommissionRate(double) void
getLastName() String
getCommissionRate() double
toString() String
getGrossSales() double
getSocialSecurityNumber() String
setGrossSales(double) void
earnings() double

EmployeeTest
EmployeeTest()
main(String[]) void
```



```
1 // Fig. 9.4: CommissionEmployee.java
2 // CommissionEmployee class represents an employee paid a
3 // percentage of gross sales.
4 public class CommissionEmployee extends Object {
5     private final String firstName;
6     private final String lastName;
7     private final String socialSecurityNumber;
8     private double grossSales; // gross weekly sales
9     private double commissionRate; // commission percentage
10
```

Fig. 9.4 | CommissionEmployee class represents an employee paid a percentage of gross sales.
(Part 1 of 5.)



```
11 // five-argument constructor
12 public CommissionEmployee(String firstName, String lastName,
13     String socialSecurityNumber, double grossSales,
14     double commissionRate) {
15     // implicit call to Object's default constructor occurs here
16
17     // if grossSales is invalid throw exception
18     if (grossSales < 0.0) {
19         throw new IllegalArgumentException("Gross sales must be >= 0.0");
20     }
21
22     // if commissionRate is invalid throw exception
23     if (commissionRate <= 0.0 || commissionRate >= 1.0) {
24         throw new IllegalArgumentException(
25             "Commission rate must be > 0.0 and < 1.0");
26     }
27
28     this.firstName = firstName;
29     this.lastName = lastName;
30     this.socialSecurityNumber = socialSecurityNumber;
31     this.grossSales = grossSales;
32     this.commissionRate = commissionRate;
33 }
```

Fig
(Pa

Fig. 9.4 | CommissionEmployee class represents an employee paid a percentage of gross sales.
(Part 2 of 5.)



```
11 // five-argument constructor
12
13 34
14 35 // return first name
15 36 public String getFirstName() {return firstName;}
16 37
17 38 // return last name
18 39 public String getLastName() {return lastName;}
19 40
20 41 // return social security number
21 42 public String getSocialSecurityNumber() {return socialSecurityNumber;}
22 43
23 44 // set gross sales amount
24 45 public void setGrossSales(double grossSales) {
25 46     if (grossSales < 0.0) {
26 47         throw new IllegalArgumentException("Gross sales must be >= 0.0");
27 48     }
28 49
29 50     this.grossSales = grossSales;
30 51 }
31
32
33
```

Fig. 9.4

(Part 1 of 5.)

Fig. 9.4 | CommissionEmployee class represents an employee paid a percentage of gross sales.

(Part 3 of 5.)

Fig

(Part 2 of 5.)



```
11 // five-argument constructor
12
13 // Fig. 9.4: CommissionEmployee class
14 // CommissionEmployee class
15 // CommissionEmployee class
16 // CommissionEmployee class
17 public class CommissionEmployee {
18 private double grossSales;
19 private double commissionRate;
20 private double grossSalesAmount;
21 private double commissionRate;
22 private double grossSalesAmount;
23 private double commissionRate;
24
25 // CommissionEmployee class
26 // CommissionEmployee class
27 // CommissionEmployee class
28 // CommissionEmployee class
29 // CommissionEmployee class
30 // CommissionEmployee class
31 // CommissionEmployee class
32 // CommissionEmployee class
33 // CommissionEmployee class
```

Fig. 9.4
(Part 1 of 5.)

```
52
53 // return gross sales amount
54 public double getGrossSales() {return grossSales;}
55
56 // set commission rate
57 public void setCommissionRate(double commissionRate) {
58     if (commissionRate <= 0.0 || commissionRate >= 1.0) {
59         throw new IllegalArgumentException(
60             "Commission rate must be > 0.0 and < 1.0");
61     }
62     this.commissionRate = commissionRate;
63 }
64
65 // return commission rate
66 public double getCommissionRate() {return commissionRate;}
67
68
```

Fig. 9.4 | CommissionEmployee class represents an employee paid a percentage of gross sales.
(Part 4 of 5.)

Fig. 9.4 | CommissionEmployee class represents an employee paid a percentage of gross sales.
(Part 4 of 5.)

Fig. 9.4 | CommissionEmployee class represents an employee paid a percentage of gross sales.
(Part 2 of 5.)



```

11 // five-argument constructor
12 public CommissionEmployee(String firstName, String lastName,
13 // Fig. 9.4: CommissionEmployee.java
14 // CommissionEmployee.java
15 // CommissionEmployee.java
16 // percentage of
17 public class CommissionEmployee {
18 private final double commissionRate;
19 private final double grossSales;
20 private final double socialSecurityNumber;
21 private double earnings;
22 private double commissionRate;
23 if (grossSales < 0) {
24 // calculate earnings
25 // calculate earnings
26 public double earnings() {return commissionRate * grossSales;}
27 // return String representation of CommissionEmployee object
28 @Override // indicates that this method overrides a superclass method
29 public String toString() {
30 return String.format("%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f",
31 "commission employee", firstName, lastName,
32 "social security number", socialSecurityNumber,
33 "gross sales", grossSales,
34 "commission rate", commissionRate);
35 }
36 }

```

Fig. 9.4
(Part 1 of 5.)

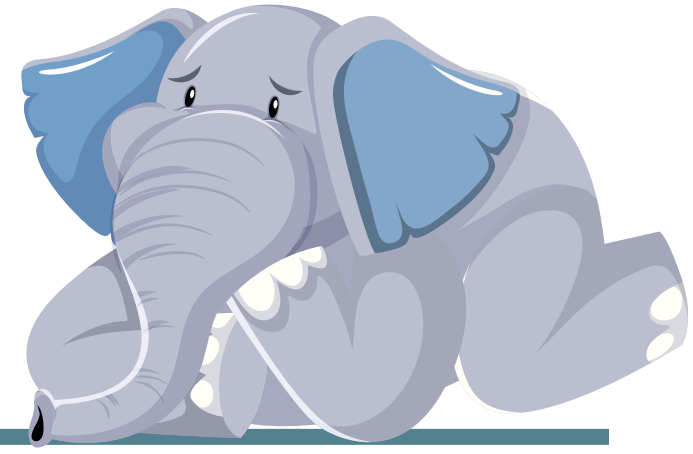
Fig. 9.4
(Part 2 of 5.)

Fig. 9.4 | CommissionEmployee class represents an employee paid a percentage of gross sales.
(Part 5 of 5.)



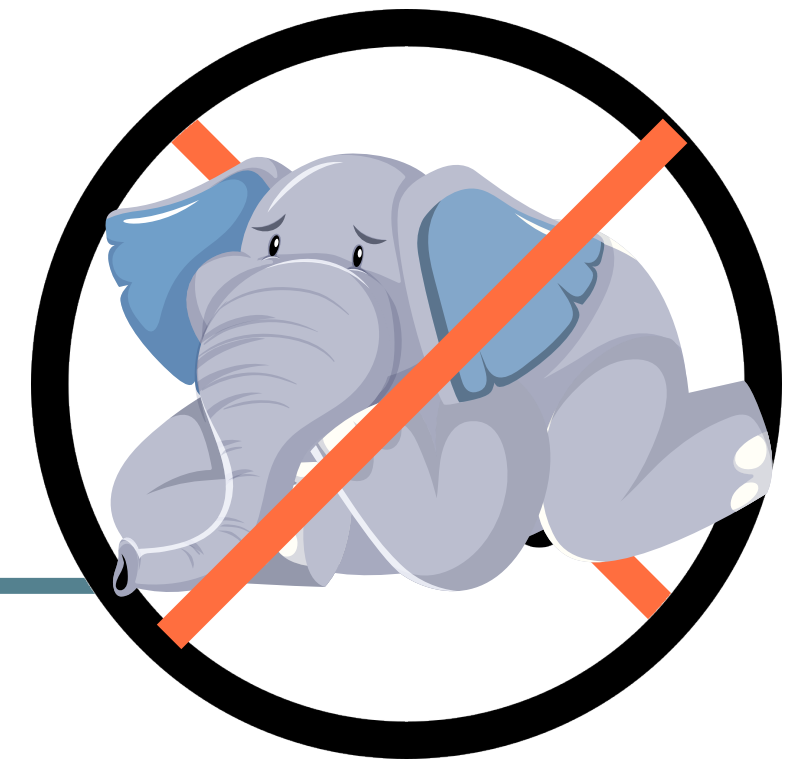
- ▶ `toString`, her sınıfın doğrudan veya dolaylı olarak `Object` sınıfından miras aldığı yöntemlerden biridir.
 - ▶ Bir nesneyi temsil eden bir `String` döndürür.
 - ▶ Bir nesnenin bir `String` temsiline dönüştürülmesi gerektiğinde örtük olarak çağrılır.
- ▶ `Object` sınıfının `toString` metodu, nesnenin sınıfının adını içeren bir `String` döndürür.
- ▶ Bu, öncelikle uygun bir `String` gösterimini belirtmek için bir alt sınıf tarafından **OVERRIDE** edilebilen bir yer tutucudur.
- ▶ Bir üst sınıf yöntemini **OVERRIDE** etmek için, bir alt sınıf, üst sınıf metoduyla aynı imzaya sahip bir metod bildirmelidir.
- ▶ `@Override` tanımlaması;
 - ▶ Bir metodun, aynı imzaya sahip bir üst sınıf yöntemini geçersiz kılması gerektiğini gösterir.
 - ▶ Bu anahtar kelime olmazsa, bir derleme hatası oluşur.

ERROR PREVENTION TIP



@Override ek açıklaması isteğe bağlı olsa da, derleme zamanında imzalarını doğru tanımladığınızdan emin olmak için geçersiz kılınan metotları onunla bildirin. Hataları çalışma zamanından ziyade derleme zamanında bulmak her zaman daha iyidir.

COMMON PROGRAMMING ERROR



Metodu geçersiz kılarken daha kısıtlı erişim değiştiricisi kullanmak bir derleme hatasıdır.

-Public bir üst sınıf metodu, protected veya private bir alt sınıf metodu olamaz;

-Protected bir üst sınıf metodu, private bir alt sınıf metodu olamaz.

Bunu yapmak, tüm alt sınıf nesnelerinin üst sınıfta bildirilen genel metotlara yapılan metot çağrılarına yanıt verebilmesini gerektiren is-a ilişkisini bozar.



```
1 // Fig. 9.5: CommissionEmployeeTest.java
2 // CommissionEmployee class test program.
3 public class CommissionEmployeeTest {
4     public static void main(String[] args) {
5         // instantiate CommissionEmployee object
6         CommissionEmployee employee = new CommissionEmployee(
7             "Sue", "Jones", "222-22-2222", 10000, .06);
8
9         // get commission employee data
10        System.out.println("Employee information obtained by get methods:");
11        System.out.printf("%n%s %s%n", "First name is",
12            employee.getFirstName());
13        System.out.printf("%s %s%n", "Last name is",
14            employee.getLastName());
15        System.out.printf("%s %s%n", "Social security number is",
16            employee.getSocialSecurityNumber());
17        System.out.printf("%s %.2f%n", "Gross sales is",
18            employee.getGrossSales());
19        System.out.printf("%s %.2f%n", "Commission rate is",
20            employee.getCommissionRate());
```

Fig. 9.5 | CommissionEmployee class test program. (Part 1 of 2.)



```
1 // Fig. 9.5: CommissionEmployeeTest.java
2 // Co
3 publi
4 pu
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22     employee.setGrossSales(5000);
23     employee.setCommissionRate(.1);
24
25     System.out.printf("%n%s:%n%n%s%n",
26         "Updated employee information obtained by toString", employee);
27 }
28 }
```

Employee information obtained by get methods:

First name is Sue
Last name is Jones
Social security number is 222-22-2222
Gross sales is 10000.00
Commission rate is 0.06

Updated employee information obtained by toString:

commission employee: Sue Jones
social security number: 222-22-2222
gross sales: 5000.00
commission rate: 0.10

Fig. 9.5 |

Fig. 9.5 | CommissionEmployee class test program. (Part 2 of 2.)



2. BasePlusCommissionEmployee:

- ▶ Doğrudan Object sınıfından miras alan.
- ▶ Bir ad, soyadı, sosyal güvenlik numarası, komisyon oranı, brüt satış tutarı ve taban maaşı gibi özel örnek değişkenlere sahip olan.
- Bu sınıfı, sınıfın gerektirdiği her kod satırını yazarak oluşturuyoruz - yakında onu, CommissionEmployee sınıfından devralarak oluşturmanın çok daha verimli olduğunu göreceğiz.

BasePlusCommissionEmployee	CommissionEmployee
BasePlusCommissionEmployee(String, String, String, double, double)	CommissionEmployee(String, String, String, double, double)
lastName: String	grossSales: double
socialSecurityNumber: String	lastName: String
firstName: String	firstName: String
commissionRate: double	socialSecurityNumber: String
baseSalary: double	commissionRate: double
grossSales: double	getFirstName(): String
getSocialSecurityNumber(): String	setCommissionRate(double): void
setCommissionRate(double): void	getLastName(): String
setBaseSalary(double): void	getCommissionRate(): double
getCommissionRate(): double	toString(): String
earnings(): double	getGrossSales(): double
setGrossSales(double): void	getSocialSecurityNumber(): String
toString(): String	setGrossSales(double): void
getLastName(): String	earnings(): double
getBaseSalary(): double	
getGrossSales(): double	
getFirstName(): String	

EmployeeTest
EmployeeTest()
main(String[]) void



- ▶ BasePlusCommissionEmployee sınıfı bir ad, soyadı, sosyal güvenlik numarası, brüt satış tutarı, komisyon oranı ve taban maaşı içerir.
- ▶ Temel maaş hariç tümü, CommissionEmployee sınıfı ile ortaktır.
- ▶ BasePlusCommissionEmployee sınıfına ait public hizmetler;
 - ▶ Constructor, her bir örnek değişkeni için bir set/get metotları ve toString metodu
- ▶ Bunların çoğu, CommissionEmployee sınıfı ile ortaktır.
- ▶ BasePlusCommissionEmployee sınıfı açıkça Object sınıfı extend edilmemiştir. (Yine de onu miras alır)



```
1 // Fig. 9.6: BasePlusCommissionEmployee.java
2 // BasePlusCommissionEmployee class represents an employee who receives
3 // a base salary in addition to commission.
4 public class BasePlusCommissionEmployee {
5     private final String firstName;
6     private final String lastName;
7     private final String socialSecurityNumber;
8     private double grossSales; // gross weekly sales
9     private double commissionRate; // commission percentage
10    private double baseSalary; // base salary per week
11 }
```

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 1 of 7.)



```
12 // six-argument constructor
13 public BasePlusCommissionEmployee(String firstName, String lastName,
14     String socialSecurityNumber, double grossSales,
15     double commissionRate, double baseSalary) {
16     // implicit call to Object's default constructor occurs here
17
18     // if grossSales is invalid throw exception
19     if (grossSales < 0.0) {
20         throw new IllegalArgumentException("Gross sales must be >= 0.0");
21     }
22
23     // if commissionRate is invalid throw exception
24     if (commissionRate <= 0.0 || commissionRate >= 1.0) {
25         throw new IllegalArgumentException(
26             "Commission rate must be > 0.0 and < 1.0");
27     }
28
```

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 2 of 7.)



```

1 // Fig. 9.6: Base
2 // BasePlusCommis
3 // a base salary
4 public class Base
5     private final
6     private final
7     private final
8     private double
9     private double
10    private double
11
22
23 // i
24 if (
25 BasePlusComm
26 salary in addition to a commis
27 }
28

```

Fig. 9.6

BasePlusComm
salary in addition to a commis

Fig. 9.6 | BaseP
salary in addition to

```

29 // if baseSalary is invalid throw exception
30 if (baseSalary < 0.0) {
31     throw new IllegalArgumentException("Base salary must be >= 0.0");
32 }
33
34 this.firstName = firstName;
35 this.lastName = lastName;
36 this.socialSecurityNumber = socialSecurityNumber;
37 this.grossSales = grossSales;
38 this.commissionRate = commissionRate;
39 this.baseSalary = baseSalary;
40 }
41
42 // return first name
43 public String getFirstName() {return firstName;}
44
45 // return last name
46 public String getLastName() {return lastName;}
47

```

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 3 of 7.)



1 // Fig. 9.6: BasePlusCommissionEmployee.java

2 // BasePlusCommissionEmployee class represents an employee who receives

3 // a base salary in addition to a commission. (Part 4 of 7.)

4 public class BasePlusCommissionEmployee {

5 private final double commissionRate;

6 private final String last;

7 private final String socialSecurityNumber;

8 private double grossSales;

9 private double baseSalary;

10 private double commissionRate;

11 private double baseSalary;

12 // if commissionRate is invalid throw exception

13 if (commissionRate < 0.0) {

14 throw new IllegalArgumentException("Commission rate must be >= 0.0");

15 }

16 }

17 }

18 }

19 }

20 }

21 }

22 }

23 }

24 }

25 }

26 }

27 }

28 }

48 // return social security number

49 public String getSocialSecurityNumber() {return socialSecurityNumber;}

50

51 // set gross sales amount

52 public void setGrossSales(double grossSales) {

53 if (grossSales < 0.0) {

54 throw new IllegalArgumentException("Gross sales must be >= 0.0");

55 }

56 this.grossSales = grossSales;

57 }

58 }

59 // return gross sales amount

60 public double getGrossSales() {return grossSales;}

61 }

62 }

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 4 of 7.)

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 4 of 7.)

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 4 of 7.)



```

1 // Fig. 9.6: BasePlusCommissionEmployee.java
2 // BasePlusCommissionEmployee class represents an employee who receives
3 // a base salary
4 public class BasePlusCommissionEmployee {
5     private final double baseSalary;
6     private final double commissionRate;
7     private final String employeeName;
8     private double grossPay;
9     private double netPay;
10    private double taxRate;
11
12    // set base salary
13    public void setBaseSalary(double baseSalary) {
14        if (baseSalary <= 0.0 || baseSalary >= 1.0) {
15            throw new IllegalArgumentException(
16                "Base salary must be > 0.0 and < 1.0");
17        }
18        this.baseSalary = baseSalary;
19    }
20
21    // set commission rate
22    public void setCommissionRate(double commissionRate) {
23        if (commissionRate <= 0.0 || commissionRate >= 1.0) {
24            throw new IllegalArgumentException(
25                "Commission rate must be > 0.0 and < 1.0");
26        }
27        this.commissionRate = commissionRate;
28    }
29
30    // return base salary
31    public double getBaseSalary() {return baseSalary;}
32
33    // return commission rate
34    public double getCommissionRate() {return commissionRate;}
35
36    // return employee name
37    public String getEmployeeName() {return employeeName;}
38
39    // calculate gross pay
40    public double calculateGrossPay() {
41        grossPay = baseSalary + (baseSalary * commissionRate);
42        return grossPay;
43    }
44
45    // calculate net pay
46    public double calculateNetPay() {
47        netPay = grossPay * (1.0 - taxRate);
48        return netPay;
49    }
50
51    // calculate tax rate
52    public double calculateTaxRate() {
53        taxRate = grossPay * 0.05;
54        return taxRate;
55    }
56
57    // calculate gross pay and net pay
58    public void calculateGrossAndNetPay() {
59        grossPay = calculateGrossPay();
60        netPay = calculateNetPay();
61    }
62
63    // set commission rate
64    public void setCommissionRate(double commissionRate) {
65        if (commissionRate <= 0.0 || commissionRate >= 1.0) {
66            throw new IllegalArgumentException(
67                "Commission rate must be > 0.0 and < 1.0");
68        }
69        this.commissionRate = commissionRate;
70    }
71
72    // return commission rate
73    public double getCommissionRate() {return commissionRate;}
74
75

```

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 5 of 7.)



```
1 // Fig. 9.6: BasePlusCommissionEmployee.java
2 // BasePlusCommissionEmployee class represents an employee who receives
3 // a base salary in addition to a commission. (Part 6 of 7.)
4 public class BasePlusCommissionEmployee {
5     private double baseSalary;
6     private double commissionRate;
7     private double commission;
8     private double grossPay;
9     private double netPay;
10    private double taxRate;
11
12    // six-argument constructor for baseSalary is invalid throw exception
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
```

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 6 of 7.)

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 6 of 7.)

```
76 // set base salary
77 public void setBaseSalary(double baseSalary) {
78     if (baseSalary < 0.0) {
79         throw new IllegalArgumentException("Base salary must be >= 0.0");
80     }
81
82     this.baseSalary = baseSalary;
83 }
84
85 // return base salary
86 public double getBaseSalary() {return baseSalary;}
87
```

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 6 of 7.)

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 4 of 7.)



```
1 // Fig.
2 // Base
3 // a ba
4 public
5     pri
6     pri
7     pri
8     pri
9     pri
10    pr
11    22
    23
    24
    25
    26
    27
    28
```

```
88 // calculate earnings
89 public double earnings() {
90     return baseSalary + (commissionRate * grossSales);
91 }
92
93 // return String representation of BasePlusCommissionEmployee
94 @Override
95 public String toString() {
96     return String.format(
97         "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f\n%s: %.2f",
98         "base-salaried commission employee", firstName, lastName,
99         "social security number", socialSecurityNumber,
100        "gross sales", grossSales, "commission rate", commissionRate,
101        "base salary", baseSalary);
102 }
103 }
```

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 7 of 7.)

Fig.
salar

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 7 of 7.)

salary in addition to a commission. (Part 3 of 7.) (Part 4 of 7.)



```
1 // Fig. 9.7: BasePlusCommissionEmployeeTest.java
2 // BasePlusCommissionEmployee test program.
3
4 public class BasePlusCommissionEmployeeTest {
5     public static void main(String[] args) {
6         // instantiate BasePlusCommissionEmployee object
7         BasePlusCommissionEmployee employee =
8             new BasePlusCommissionEmployee(
9                 "Bob", "Lewis", "333-33-3333", 5000, .04, 300);
10
11         // get base-salaried commission employee data
12         System.out.printf(
13             "Employee information obtained by get methods:%n");
14         System.out.printf("%s %s%n", "First name is",
15             employee.getFirstName());
```

Fig. 9.7 | BasePlusCommissionEmployee test program. (Part 1 of 3.)



```
1 // Fig
2 // Bas
3
4 public
5     pub
6
7
8
9
10
11
12
13
14
15
```

Fig. 9.7 | B

Copyright © 2020 Pearson

```
16      System.out.printf("%s %s\n", "Last name is",
17                          employee.getLastName());
18      System.out.printf("%s %s\n", "Social security number is",
19                          employee.getSocialSecurityNumber());
20      System.out.printf("%s %.2f\n", "Gross sales is",
21                          employee.getGrossSales());
22      System.out.printf("%s %.2f\n", "Commission rate is",
23                          employee.getCommissionRate());
24      System.out.printf("%s %.2f\n", "Base salary is",
25                          employee.getBaseSalary());
26
27      employee.setBaseSalary(1000);
28
29      System.out.printf("%n%s:%n%n%s\n",
30                          "Updated employee information obtained by toString",
31                          employee.toString());
32  }
33 }
```

Fig. 9.7 | BasePlusCommissionEmployee test program. (Part 2 of 3.)



```
1 // Fig. 9.7: BasePlusCommissionEmployeeTest.java
2 // BasePlusCommissionEmployee test program
3
4 public class BasePlusCommissionEmployeeTest {
5     public static void main(String[] args) {
6         // instantiate a BasePlusCommissionEmployee object
7         BasePlusCommissionEmployee employee =
8             new BasePlusCommissionEmployee(
9                 "Bob", "Lewis", "333-33-3333",
10                 5000.00, 0.04, 300.00);
11
12         // get base-salaried commission employee information
13         System.out.println("Employee information obtained by get methods:");
14         System.out.println("First name is " + employee.getFirstName());
15         System.out.println("Last name is " + employee.getLastName());
16         System.out.println("Social security number is " + employee.getSocialSecurityNumber());
17         System.out.println("Gross sales is " + employee.getGrossSales());
18         System.out.println("Commission rate is " + employee.getCommissionRate());
19         System.out.println("Base salary is " + employee.getBaseSalary());
20
21         // update employee information
22         employee.setGrossSales(10000.00);
23         employee.setCommissionRate(0.05);
24         employee.setBaseSalary(1000.00);
25
26         // get base-salaried commission employee information
27         System.out.println("Updated employee information obtained by toString:");
28         System.out.println(employee.toString());
29     }
30 }
31
32
33
```

Fig. 9.7 | BasePlusCommissionEmployee test program. (Part 3 of 3.)

Copyright © 2020 Pearson Education Ltd. All Rights Reserved.

Employee information obtained by get methods:

First name is Bob

Last name is Lewis

Social security number is 333-33-3333

Gross sales is 5000.00

Commission rate is 0.04

Base salary is 300.00

Updated employee information obtained by toString:

base-salaried commission employee: Bob Lewis

social security number: 333-33-3333

gross sales: 5000.00

commission rate: 0.04

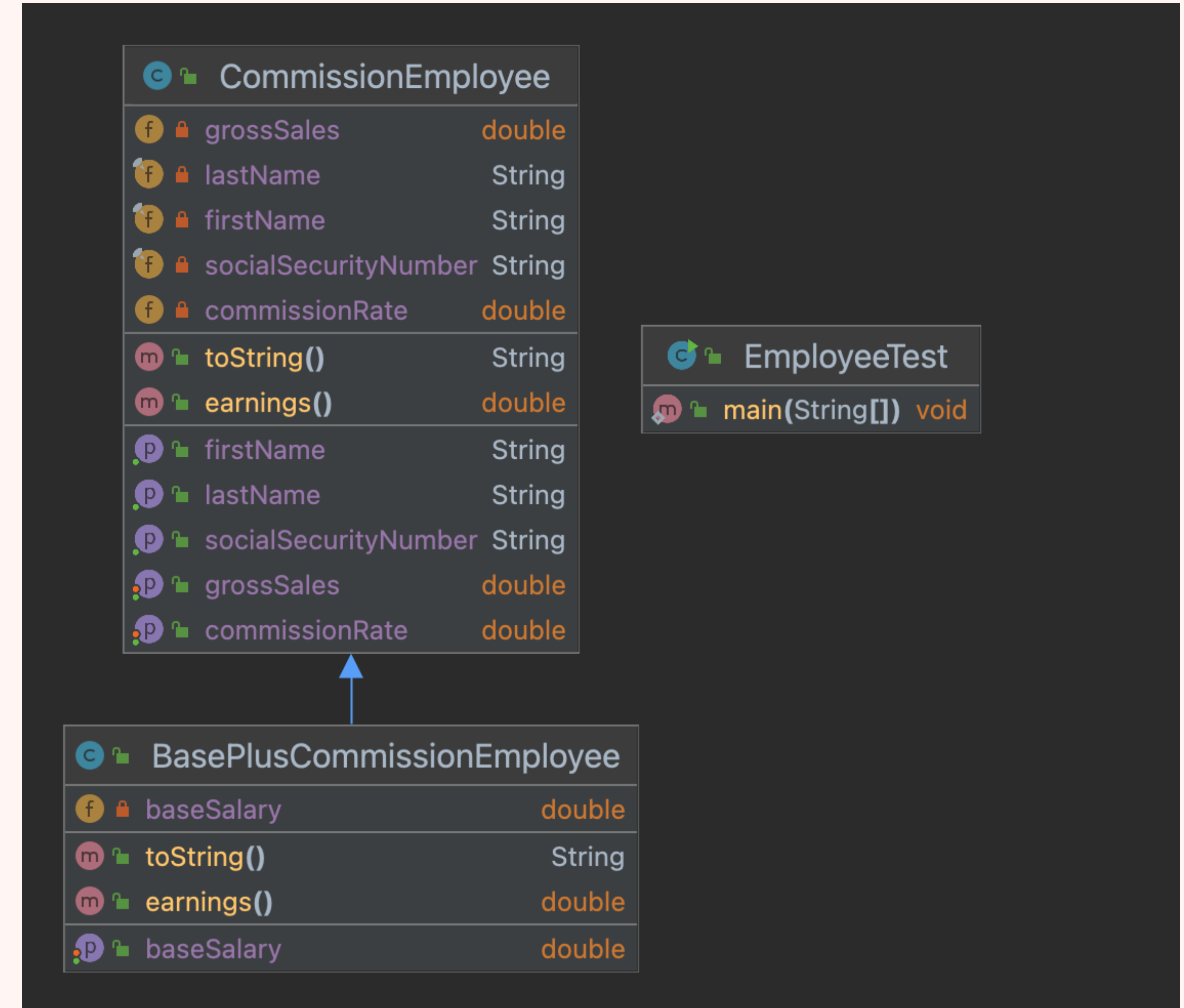
base salary: 1000.00

Fig. 9.7 | BasePlusCommissionEmployee test program. (Part 3 of 3.)



3. BasePlusCommissionEmployee:

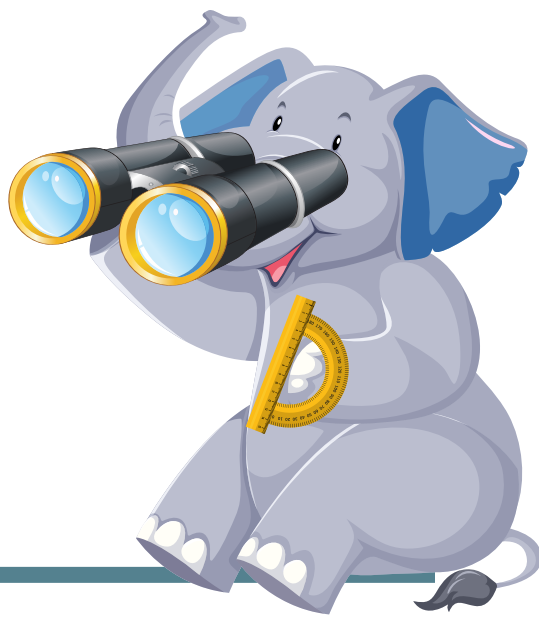
- ▶ **CommissionEmployee** sınıfını miras alan
- ▶ Bu durum yazılımın yeniden kullanımı örneğidir.
- ▶ Yeni alt sınıfı geliştirirken çok daha az kod yazmamızı sağlar.
- ▶ Constructor içinde "super" ile üst sınıfa erişilir.
- ▶ Bu örnekte, BasePlusCommissionEmployee sınıfı, CommissionEmployee sınıfının özel üyelerine erişmeye çalışır;
- ▶ Alt sınıf üst sınıfın özel örnek değişkenlerine erişemediğinden derleme hatalarına neden olur.





- ▶ İki sınıf da benzer örnek değişkenler ve metotlar içerir
 - ▶ Temelde, Bir sınıfı kopyalayıp, baseSalary eklemek yeni sınıfı oluşturur.
 - ▶ Bu yöntem zaman kayıplarına ve hatalara sebep olabilir.
 - ▶ Uzun vadede bakım problemleri de ortaya çıkacaktır.

SOFTWARE ENGINEERING OBSERVATION



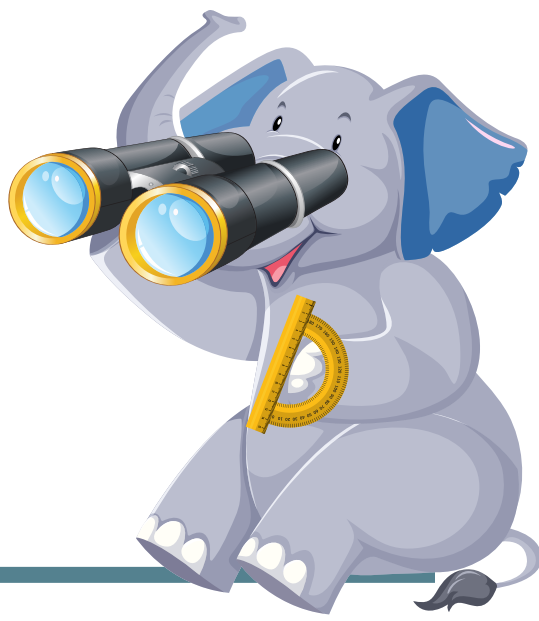
Kalıtım ile, hiyerarşideki tüm sınıflar için aynı olan örnek değişkenler ve yöntemler bir üst sınıfta bildirilir. Üst sınıftaki bu ortak özelliklerde yapılan değişiklikler, alt sınıf tarafından miras alınır.

Kalıtım olmadan, söz konusu kodun bir kopyasını içeren tüm kaynak kod dosyalarında değişiklik yapılması gerekir.



- ▶ BasePlusCommissionEmployee, CommissionEmployee sınıfını miras alır.
- ▶ Kalıtım bu sınıfa CommissionEmployee sınıfının tüm yeteneklerini geçirir.
- ▶ Burada sadece baseSalary örnek değişkeni eklenecektir.
- ▶ Alt sınıf CommissionEmployee sınıfının sadece “public ve protected” üyelerine erişebilir.

SOFTWARE ENGINEERING OBSERVATION



Nesne yönelimli bir sistemin tasarım aşamasında, genellikle belirli sınıfların birbirleriyle yakından ilişkili olduğunu görürsünüz.

Ortak örnek değişkenlerini ve metotları "dışarıya ayırmalı" ve bunları bir üst sınıfa yerleştirmelisiniz.

Ardından, üst sınıftan devralınanların ötesinde yeteneklerle onları uzmanlaştırarak alt sınıflar geliştirmek için kalıtımı kullanın.

SOFTWARE ENGINEERING OBSERVATION

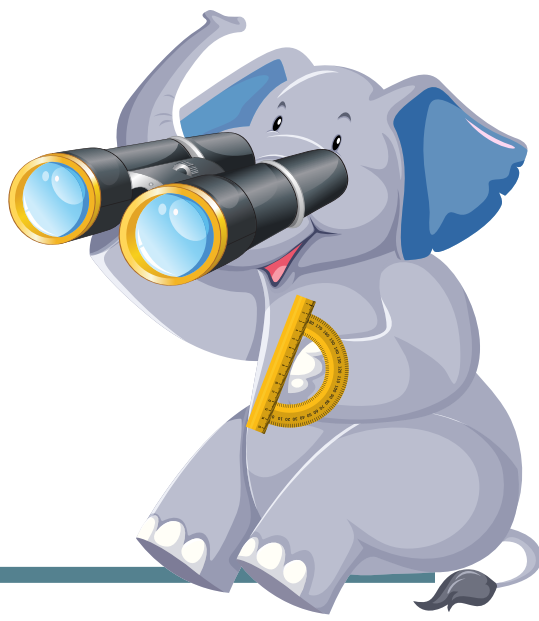


Nesne yönelimli bir sistemin tasarım aşamasında, genellikle belirli sınıfların birbirleriyle yakından ilişkili olduğunu görürsünüz.

Ortak örnek değişkenlerini ve metotları "dışarıya ayırmalı" ve bunları bir üst sınıfa yerleştirmelisiniz.

Ardından, üst sınıftan devralınanların ötesinde yeteneklerle onları uzmanlaştırarak alt sınıflar geliştirmek için kalıtımı kullanın.

SOFTWARE ENGINEERING OBSERVATION



Bir alt sınıfı bildirmek, onun üst sınıfının kaynak kodunu etkilemez.

Kalıtım, üst sınıfın bütünlüğünü korur.



```
1 // Fig. 9.8: BasePlusCommissionEmployee.java
2 // private superclass members cannot be accessed in a subclass.
3 public class BasePlusCommissionEmployee extends CommissionEmployee {
4     private double baseSalary; // base salary per week
5
6     // six-argument constructor
7     public BasePlusCommissionEmployee(String firstName, String lastName,
8         String socialSecurityNumber, double grossSales,
9         double commissionRate, double baseSalary) {
10         // explicit call to superclass CommissionEmployee constructor
11         super(firstName, lastName, socialSecurityNumber,
12             grossSales, commissionRate);
13
14         // if baseSalary is invalid throw exception
15         if (baseSalary < 0.0) {
16             throw new IllegalArgumentException("Base salary must be >= 0.0");
17         }
18
19         this.baseSalary = baseSalary;
20     }
```

Fig. 9.8 | private superclass members cannot be accessed in a subclass. (Part I of 5.)



```
1 // Fig. 9.8: BasePlusCommissionEmployee.java
2 // pr
3 publi
4 pr
5
6 //
7 pul
8
9
10
11
12
13
14
15
16
17
18
19
20 }
```

```
21
22 // set base salary
23 public void setBaseSalary(double baseSalary) {
24     if (baseSalary < 0.0) {
25         throw new IllegalArgumentException("Base salary must be >= 0.0");
26     }
27
28     this.baseSalary = baseSalary;
29 }
30
31 // return base salary
32 public double getBaseSalary() {return baseSalary;}
33
```

Fig. 9.8 | private superclass members cannot be accessed in a subclass. (Part 2 of 5.)

Fig. 9.8 | private superclass members cannot be accessed in a subclass. (Part 1 of 5.)



```

1 // Fig. 9.8: BasePlusCommissionEmployee.java
2 // private superclass members cannot be accessed in subclass
3 public class BasePlusCommissionEmployee {
4     private double baseSalary;
5     private double commissionRate;
6     // six-digit social security number
7     public String socialSecurityNumber;
8     String firstName;
9     double grossSales;
10    // earnings
11    // superclass
12    // gross sales
13    // commission rate
14    // if (grossSales < 0)
15    if (grossSales < 0)
16        throw new IllegalArgumentException("Gross sales cannot be negative");
17 }
18
19 // Fig. 9.8: BasePlusCommissionEmployee.java
20 }
21
22 // calculate earnings
23 @Override
24 public double earnings() {
25     // not allowed: commissionRate and grossSales private in superclass
26     return baseSalary + (commissionRate * grossSales);
27 }
28
29 // return String representation of BasePlusCommissionEmployee
30 @Override
31 public String toString() {
32     // not allowed: attempts to access private superclass members
33     return String.format(
34         "%s: %s %s%n%s: %s%n%s: %.2f%n%s: %.2f%n%s: %.2f",
35         "base-salaried commission employee", firstName, lastName,
36         "social security number", socialSecurityNumber,
37         "gross sales", grossSales, "commission rate", commissionRate,
38         "base salary", baseSalary);
39 }
40 }

```

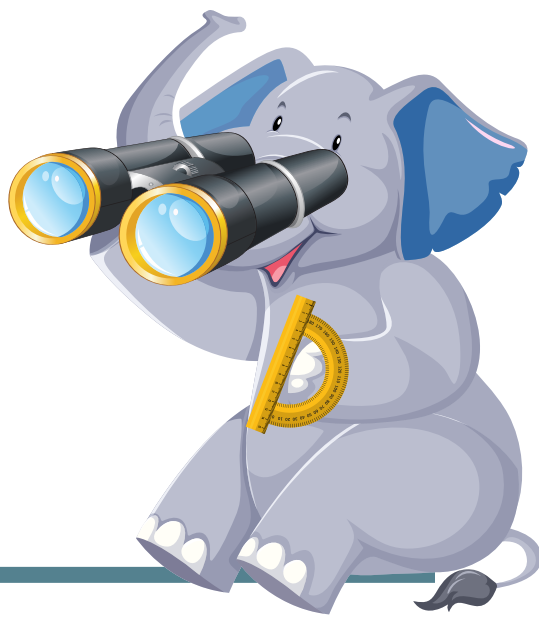
Fig. 9.8 | private superclass members cannot be accessed in a subclass. (Part 3 of 5.)

Fig. 9.8 | private superclass members cannot be accessed in a subclass. (Part 3 of 5.)



- ▶ Bir alt sınıf constructor için ilk görev, doğrudan üst sınıfının constructor açık veya örtük olarak çağırmaktır.
- ▶ Üst sınıftan devralınan örnek değişkenlerinin düzgün bir şekilde başlatılmasını sağlar.
- ▶ Üst sınıf constructor çağırısı: “super” anahtar sözcüğü ardından üst sınıf örnek değişkenleri parantez içinde verilir.
- ▶ Constructor gövdesindeki ilk ifade “super” olmalıdır.
- ▶ Constructor miras alınmaz.
- ▶ Kod, üst sınıf constructor için açık bir çağrı içermiyorsa, Java tarafından, üst sınıfın varsayılan veya argümansız Constructor dolaylı olarak çağırılır.

SOFTWARE ENGINEERING OBSERVATION



**Daha önce bir sınıfın örnek yöntemlerini constructor dan
çağdırmamanız gerektiğini öğrendiniz ve bunun nedenini 5.
Bölümde söyleyeceğiz.**

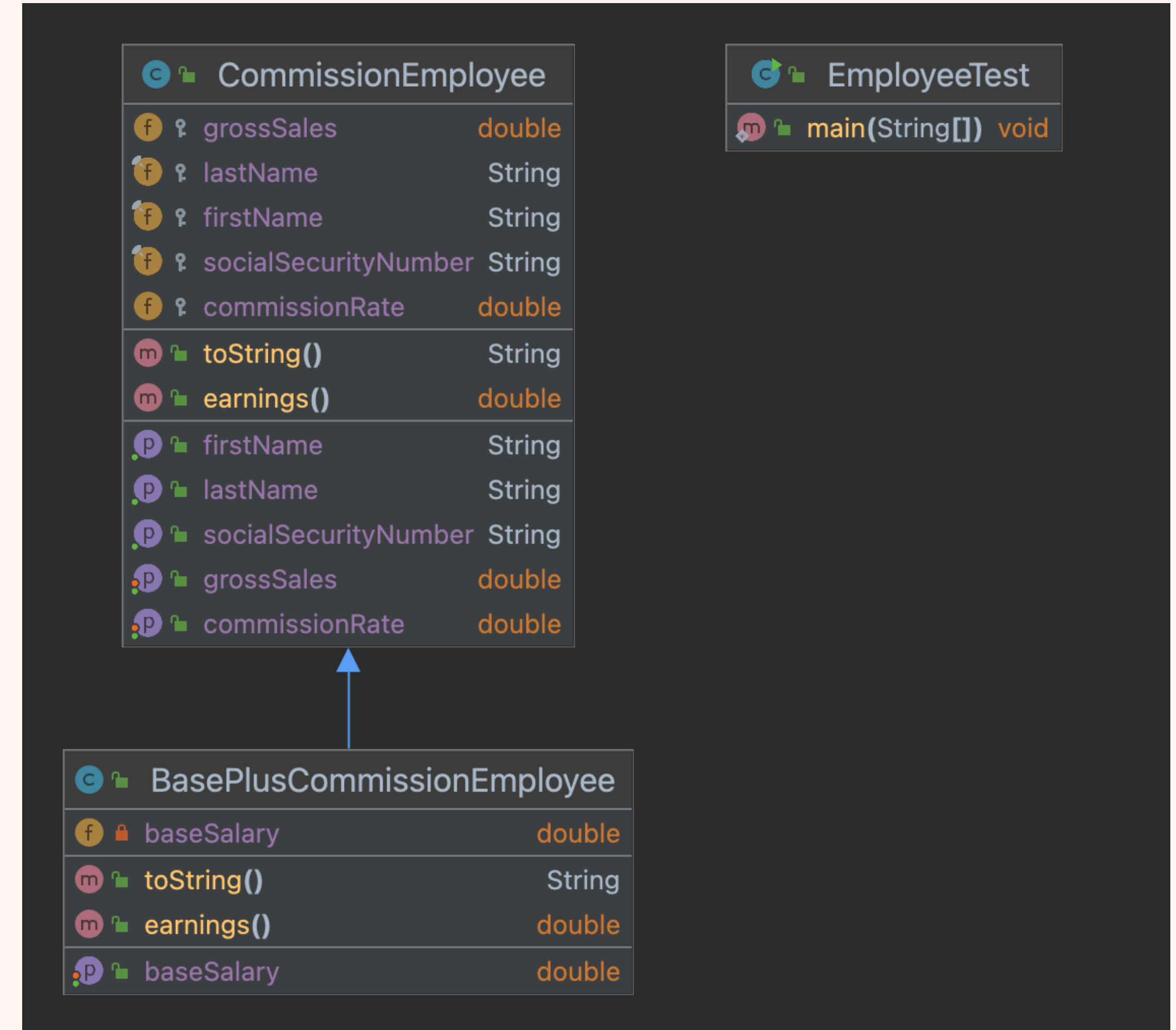
**Bir alt sınıf kurucusundan bir üst sınıf kurucusunu
çağdırmak bu tavsiyeyle çelişmez.**

Delegate constructor çağdırmak gibi düşünülebilir.



4. Protected Örnek Değişkenler:

- ▶ Protected üyelere alt sınıflar erişebilir.
- ▶ Alt sınıf olmayan ve farklı pakette olan diğer sınıflar erişemez.
- ▶ CommissionEmployee'nin örnek değişkenleri "protected" olarak bildirilirse BasePlusCommissionEmployee alt sınıfının bu verilere doğrudan erişebileceğini gösterir.
- ▶ Her iki BasePlusCommissionEmployee sınıfı da aynı işlevselliğe sahiptir, ancak devralınan sürümün nasıl daha kolay oluşturulacağını ve yönetileceğini gösteriyoruz.
- ▶ BasePlusCommisionEmployee sınıfından türeyen bir başka sınıf da yine CommissionEmployee protected değişkenlerine erişebilir.





```
1 // Fig. 9.9: BasePlusCommissionEmployee.java
2 // BasePlusCommissionEmployee inherits protected instance
3 // variables from CommissionEmployee.
4
5 public class BasePlusCommissionEmployee extends CommissionEmployee {
6     private double baseSalary; // base salary per week
7
8     // six-argument constructor
9     public BasePlusCommissionEmployee(String firstName, String lastName,
10         String socialSecurityNumber, double grossSales,
11         double commissionRate, double baseSalary) {
12         super(firstName, lastName, socialSecurityNumber,
13             grossSales, commissionRate);
14
15         // if baseSalary is invalid throw exception
16         if (baseSalary < 0.0) {
17             throw new IllegalArgumentException("Base salary must be >= 0.0");
18         }
19
20         this.baseSalary = baseSalary;
21     }
```

Fig. 9.9 | BasePlusCommissionEmployee inherits protected instance variables from CommissionEmployee. (Part I of 3.)



```
1 // Fig. 9.9: BasePlusCommissionEmployee.java
2 // E
3 // v
4
5 publ
6 p
7
8 /
9 p
10
11
12
13
14
15
16
17
18
19
20
21
```

```
22
23 // set base salary
24 public void setBaseSalary(double baseSalary) {
25     if (baseSalary < 0.0) {
26         throw new IllegalArgumentException("Base salary must be >= 0.0");
27     }
28
29     this.baseSalary = baseSalary;
30 }
31
32 // return base salary
33 public double getBaseSalary() {return baseSalary;}
34
```

Fig. 9.9 | BasePlusCommissionEmployee inherits protected instance variables from CommissionEmployee. (Part 2 of 3.)

Fig. 9.9 | BasePlusCommissionEmployee inherits protected instance variables from CommissionEmployee. (Part 1 of 3.)



```

1 // Fig. 9.9: BasePlusCommissionEmployee.java
2 // BasePlusCommissionEmployee
3 // variables from CommissionEmployee
4     22
5 public class BasePlusCommissionEmployee {
6     private double baseSalary;
7     25
8     // six argument constructor
9     public BasePlusCommissionEmployee(String firstName,
10        String socialSecurityNumber, double commissionRate,
11        double grossSales, String last, String middle,
12        String first) {
13         super(firstName, socialSecurityNumber, last, middle, first);
14         32 // set base salary
15         33 if (baseSalary < 0)
16             34 throw new IllegalArgumentException("base salary must be non-negative");
17     }
18     Fig. 9.9 | BasePlusCommissionEmployee
19     CommissionEmployee
20     this.baseSalary = baseSalary;
21 }

```

Fig. 9.9 | BasePlusCommissionEmployee inherits protected instance variables from CommissionEmployee. (Part 1 of 3.)

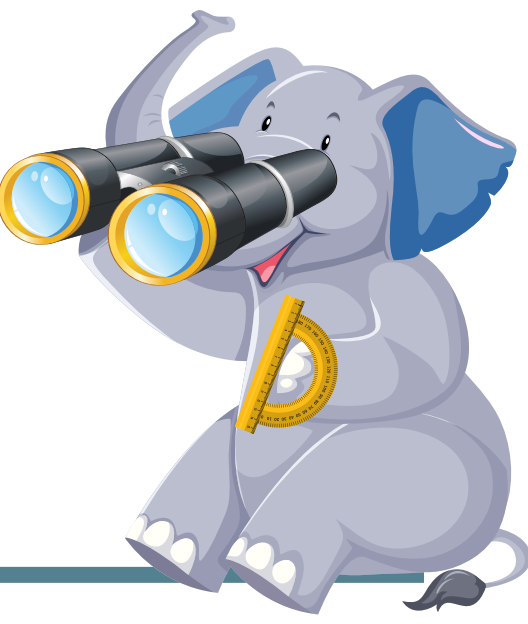
```

35 // calculate earnings
36 @Override // indicates that this method overrides a superclass method
37 public double earnings() {
38     return baseSalary + (commissionRate * grossSales);
39 }
40
41 // return String representation of BasePlusCommissionEmployee
42 @Override
43 public String toString() {
44     return String.format(
45         "%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f\n%s: %.2f",
46         "base-salaried commission employee", firstName, lastName,
47         "social security number", socialSecurityNumber,
48         "gross sales", grossSales, "commission rate", commissionRate,
49         "base salary", baseSalary);
50 }
51 }

```

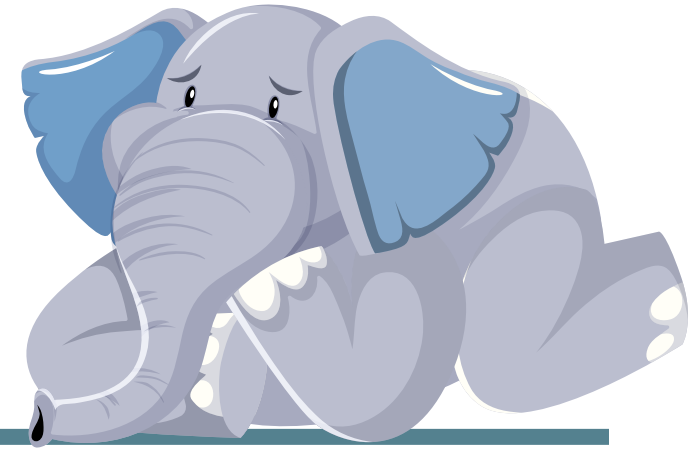
Fig. 9.9 | BasePlusCommissionEmployee inherits protected instance variables from CommissionEmployee. (Part 3 of 3.)

SOFTWARE ENGINEERING OBSERVATION



Üst sınıf örnek değişkenlerini private (protected yerine) bildirmek, bu örnek değişkenlerinin üst sınıf uygulamasının alt sınıf uygulamalarını etkilemeden değişmesini sağlar.

ERROR PREVENTION TIP



Protected örnek değişkenlerinden kaçının.

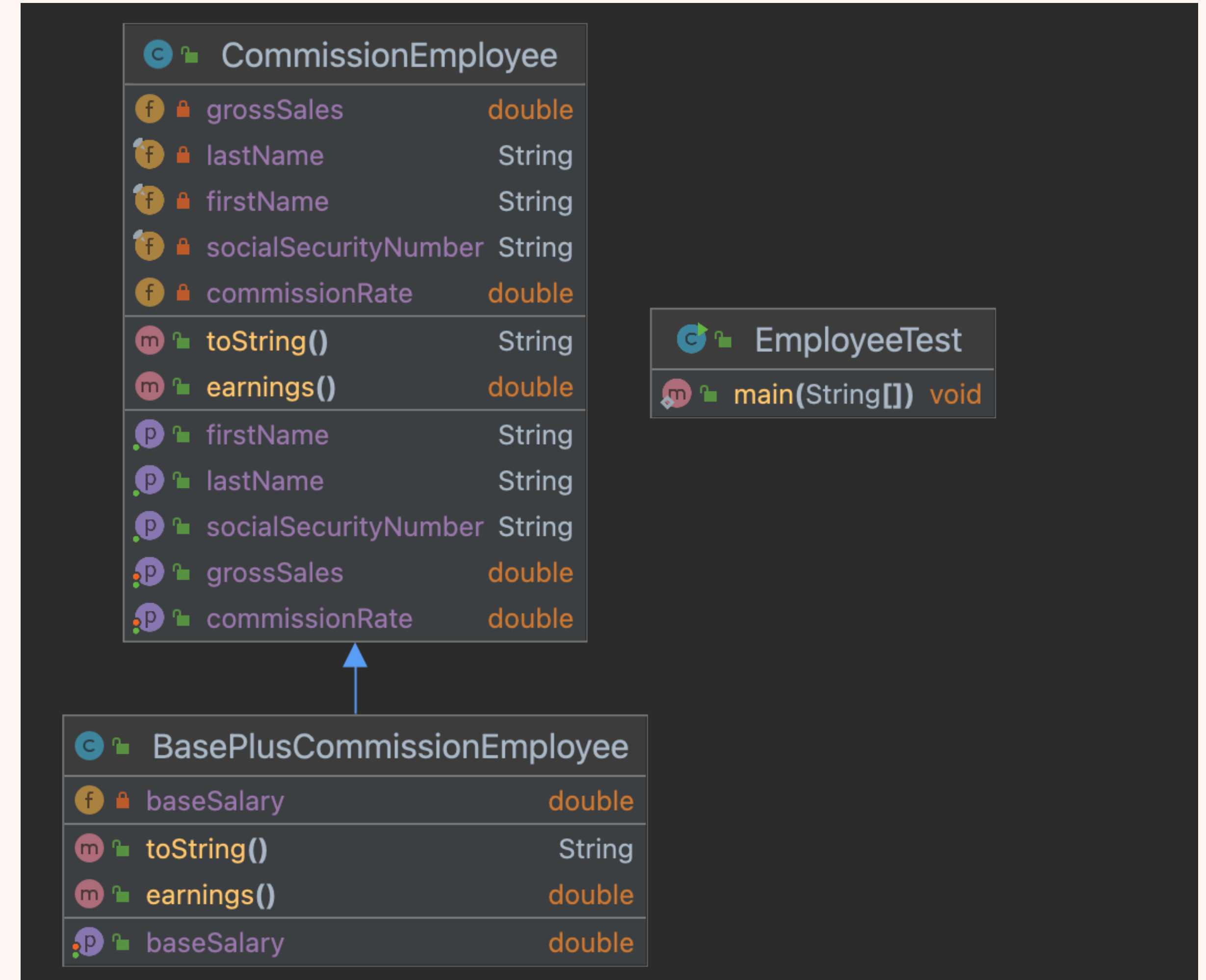
Bunun yerine, private örnek değişkenlerine erişen özel olmayan yöntemleri yazın.

Bu, sınıftaki nesnelerin tutarlı durumları sürdürmesini sağlamaya yardımcı olacaktır.



5. Get-set metotlar:

- BasePlusCommissionEmployee alt sınıfının, CommissionEmployee'den devralınan özel örnek değişkenlerini (kontrollü bir şekilde) değiştirmek için CommissionEmployee'nin genel yöntemlerini kullanır. (set-get)
- toString ve earnings metotları için get-metotları kullanılır.
- Üst sınıf yöntemlere erişmek için "super" anahtar kelimesi kullanılır.





```
1 // Fig. 9.10: CommissionEmployee.java
2 // CommissionEmployee class uses methods to manipulate its
3 // private instance variables.
4 public class CommissionEmployee {
5     private final String firstName;
6     private final String lastName;
7     private final String socialSecurityNumber;
8     private double grossSales; // gross weekly sales
9     private double commissionRate; // commission percentage
10
11     // five-argument constructor
12     public CommissionEmployee(String firstName, String lastName,
13         String socialSecurityNumber, double grossSales,
14         double commissionRate) {
15         // implicit call to Object constructor occurs here
16
17         // if grossSales is invalid throw exception
18         if (grossSales < 0.0) {
19             throw new IllegalArgumentException("Gross sales must be >= 0.0");
20         }
21     }
22 }
```

Fig. 9.10 | CommissionEmployee class uses methods to manipulate its private instance variables. (Part 1 of 5.)



```
1 // Fig. 9.10: CommissionEmployee.java
2 // CommissionEmployee class uses methods to manipulate its
3 // private instance variables. (Part 2 of 5.)
4 public class CommissionEmployee {
5     private String firstName;
6     private String lastName;
7     private double socialSecurityNumber;
8     private double grossSales;
9     private double commissionRate;
10
11     // Setters
12     public void setFirstName(String firstName) {
13         // if commissionRate is invalid throw exception
14         if (commissionRate <= 0.0 || commissionRate >= 1.0) {
15             throw new IllegalArgumentException(
16                 "Commission rate must be > 0.0 and < 1.0");
17         }
18         this.firstName = firstName;
19     }
20     public void setLastName(String lastName) {
21         this.lastName = lastName;
22     }
23     public void setSocialSecurityNumber(double socialSecurityNumber) {
24         this.socialSecurityNumber = socialSecurityNumber;
25     }
26     public void setGrossSales(double grossSales) {
27         this.grossSales = grossSales;
28     }
29     public void setCommissionRate(double commissionRate) {
30         this.commissionRate = commissionRate;
31     }
32 }
33
34
```

Fig. 9.10 |
variables. (Part

Fig. 9.10 | CommissionEmployee class uses methods to manipulate its private instance variables. (Part 2 of 5.)



```

1 // Fig. 9.10: Commission
2 // CommissionEmployee cl
3 // private instance vari
4 public class CommissionE
5     private final String
6     private final String
7     private final String
8     private double grossS
9     private double commis
10
11 // five-argument cons
12 public CommissionEmp
13     String socialSecur
14     double commissionR
15 // implicit call t
16
17 // if grossSales i
18 if (grossSales < 0
19     throw new Illeg
20 }

```

Fig. 9.10 | CommissionEmployee variables. (Part 1 of 5.)

Fig. 9.10 | variables. (Part

```

35 // return first name
36 public String getFirstName() {return firstName;}
37
38 // return last name
39 public String getLastName() {return lastName;}
40
41 // return social security number
42 public String getSocialSecurityNumber() {return socialSecurityNumber;}
43
44 // set gross sales amount
45 public void setGrossSales(double grossSales) {
46     if (grossSales < 0.0) {
47         throw new IllegalArgumentException("Gross sales must be >= 0.0");
48     }
49
50     this.grossSales = grossSales;
51 }
52
53 // return gross sales amount
54 public double getGrossSales() {return grossSales;}

```

Fig. 9.10 | CommissionEmployee class uses methods to manipulate its private instance variables. (Part 3 of 5.)



```

1 // Fig. 9.10: CommissionEmployee.java
2 // CommissionEmployee class uses methods to manipulate its
3 // private instance variables.
4 public class CommissionEmployee {
5     private final String firstName; //
6     private final String lastName; //
7     private final String socialSecurityNumber; //
8     private double grossSales; // gross sales
9     private double commissionRate; // commission rate
10
11 // five-argument constructor
12 public CommissionEmployee(String firstName, String lastName, String socialSecurityNumber, double grossSales, double commissionRate) {
13     // implicit call to Object constructor
14     this.firstName = firstName;
15     this.lastName = lastName;
16     this.socialSecurityNumber = socialSecurityNumber;
17     // if grossSales is negative, throw an exception
18     if (grossSales < 0) {
19         throw new IllegalArgumentException("Gross sales cannot be negative.");
20     }
21 }

```

Fig. 9.10 | CommissionEmployee class uses methods to manipulate its private instance variables. (Part 1 of 5.)

Fig. 9.10 | CommissionEmployee class uses methods to manipulate its private instance variables. (Part 2 of 5.)

Fig. 9.10 | CommissionEmployee class uses methods to manipulate its private instance variables. (Part 3 of 5.)

```

55
56 // set commission rate
57 public void setCommissionRate(double commissionRate) {
58     if (commissionRate <= 0.0 || commissionRate >= 1.0) {
59         throw new IllegalArgumentException(
60             "Commission rate must be > 0.0 and < 1.0");
61     }
62
63     this.commissionRate = commissionRate;
64 }
65
66 // return commission rate
67 public double getCommissionRate() {return commissionRate;}
68

```

Fig. 9.10 | CommissionEmployee class uses methods to manipulate its private instance variables. (Part 4 of 5.)



```

1 // Fig. 9.10: CommissionEmployee.java
2 // CommissionEmployee class uses methods to manipulate its
3 // private instance
4 public class Commis
5     private final Str
6     private final Str
7     private final Str
8     private double gr
9     private double co
10
11 // five-argument
12 public Commission
13     String socialS
14     double commiss
15     // implicit ca
16
17 // if grossSal
18 if (grossSales
19     then new I
20 }
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59 // calculate earnings
60 public double earnings() {
61     return getCommissionRate() * getGrossSales();
62 }
63
64
65
66
67
68
69 // return String representation of CommissionEmployee object
70 @Override
71 public String toString() {
72     return String.format("%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f",
73         "commission employee", getFirstName(), getLastName(),
74         "social security number", getSocialSecurityNumber(),
75         "gross sales", getGrossSales(),
76         "commission rate", getCommissionRate());
77 }
78
79
80
81
82
83

```

Fig. 9.10 | CommissionEmployee class uses methods to manipulate its private instance variables. (Part 1 of 5.)

Fig. 9.10 | CommissionEmployee class uses methods to manipulate its private instance variables. (Part 5 of 5.)

Fig. 9.10 | CommissionEmployee class uses methods to manipulate its private instance variables. (Part 5 of 5.)

stance



```
1 // Fig. 9.11: BasePlusCommissionEmployee.java
2 // BasePlusCommissionEmployee class inherits from CommissionEmployee
3 // and accesses the superclass's private data via inherited
4 // public methods.
5 public class BasePlusCommissionEmployee extends CommissionEmployee {
6     private double baseSalary; // base salary per week
7 }
```

Fig. 9.11 | BasePlusCommissionEmployee class inherits from CommissionEmployee and accesses the superclass's private data via inherited public methods. (Part 1 of 4.)



```
1 // Fig. 9.11: BasePlusCommissionEmployee.java
```

```
2 // B
```

```
3 // a
```

```
4 // p
```

```
5 publ
```

```
6 p
```

```
7
```

Fig. 9.11

accesses the

```
8 // six-argument constructor
9 public BasePlusCommissionEmployee(String firstName, String lastName,
10 String socialSecurityNumber, double grossSales,
11 double commissionRate, double baseSalary) {
12     super(firstName, lastName, socialSecurityNumber,
13         grossSales, commissionRate);
14
15     // if baseSalary is invalid throw exception
16     if (baseSalary < 0.0) {
17         throw new IllegalArgumentException("Base salary must be >= 0.0");
18     }
19
20     this.baseSalary = baseSalary;
21 }
22
```

Fig. 9.11 | BasePlusCommissionEmployee class inherits from CommissionEmployee and accesses the superclass's private data via inherited public methods. (Part 2 of 4.)



```
1 // Fig. 9.11: BasePlusCommissionEmployee.java
2 // BasePlusCommissionEmployee class inherits from CommissionEmployee
3 // and accesses the superclass's private data via inherited
4 // public methods// six-argument constructor
5 public class BasePlusCommissionEmployee {String firstName, String lastName,
6     private double
7     11
8     12
9     13
10    14
11    15
12    16
13    17
14    18
15    19
16    20
17    21
18    22
```

Fig. 9.11 | BasePlusCom
accesses the superclass's pr

```
23 // set base salary
24 public void setBaseSalary(double baseSalary) {
25     if (baseSalary < 0.0) {
26         throw new IllegalArgumentException("Base salary must be >= 0.0");
27     }
28
29     this.baseSalary = baseSalary;
30 }
31
32 // return base salary
33 public double getBaseSalary() {return baseSalary;}
34
```

Fig. 9.11
accesses th

Fig. 9.11 | BasePlusCommissionEmployee class inherits from CommissionEmployee and accesses the superclass's private data via inherited public methods. (Part 3 of 4.)



```

1 // Fig. 9.11: BasePlusCommissionEmployee.java
2 // BasePlusCommissionEmployee class inherits from CommissionEmployee
3 // and accesses the superclass's private data via inherited
4 // public methods // six-argument constructor
5 public class BasePlusCommissionEmployee {
6     private String firstName, String lastName,
7         String socialSecurityNumber;
8     private double grossSales,
9         double commissionRate, double baseSalary;
10    BasePlusCommissionEmployee(String firstName, String lastName,
11        String socialSecurityNumber, double grossSales,
12        double commissionRate, double baseSalary) {
13        super(firstName, lastName, socialSecurityNumber);
14        this.grossSales = grossSales;
15        this.commissionRate = commissionRate;
16        this.baseSalary = baseSalary;
17    }
18    // accessor methods
19    public double getGrossSales() const { return grossSales; }
20    public void setGrossSales(double newGrossSales) {
21        grossSales = newGrossSales;
22    }
23    public double getCommissionRate() const { return commissionRate; }
24    public void setCommissionRate(double newCommissionRate) {
25        commissionRate = newCommissionRate;
26    }
27    public double getBaseSalary() const { return baseSalary; }
28    public void setBaseSalary(double newBaseSalary) {
29        baseSalary = newBaseSalary;
30    }
31    // calculate earnings
32    @Override
33    public double earnings() const { return getBaseSalary() + super.earnings(); }
34    // return String representation of BasePlusCommissionEmployee
35    @Override
36    public String toString() const {
37        return String.format("%s %s\n%s: %.2f", "base-salaried",
38            super.toString(), "base salary", getBaseSalary());
39    }
40 }

```

Fig. 9.11 | BasePlusCommissionEmployee class inherits from CommissionEmployee and accesses the superclass's private data via inherited public methods.

Fig. 9.11 | BasePlusCommissionEmployee class inherits from CommissionEmployee and accesses the superclass's private data via inherited public methods.

```

35 // calculate earnings
36 @Override
37 public double earnings() { return getBaseSalary() + super.earnings(); }
38
39 // return String representation of BasePlusCommissionEmployee
40 @Override
41 public String toString() {
42     return String.format("%s %s\n%s: %.2f", "base-salaried",
43         super.toString(), "base salary", getBaseSalary());
44 }
45 }

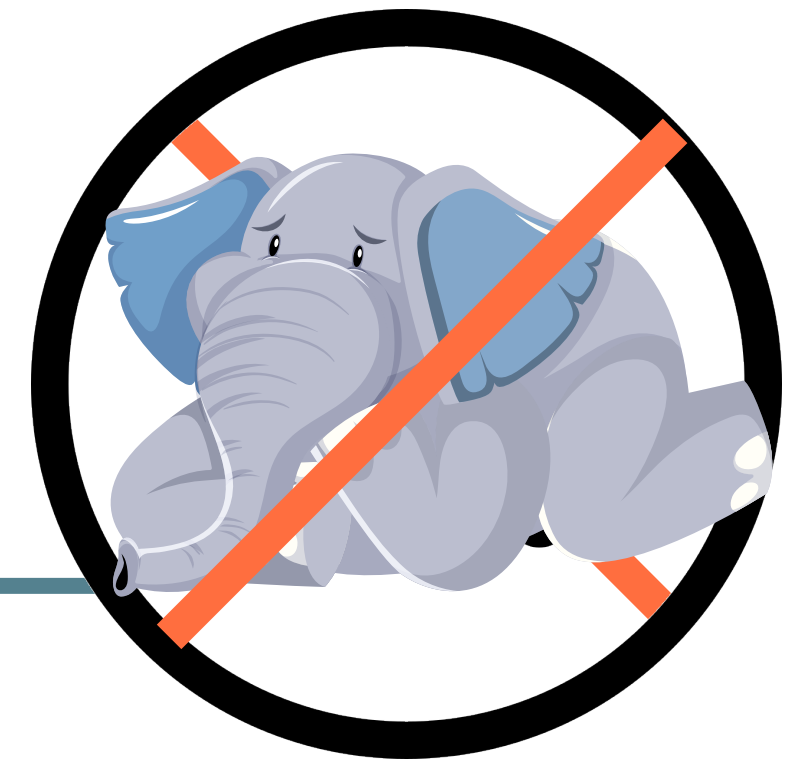
```

Fig. 9.11 | BasePlusCommissionEmployee class inherits from CommissionEmployee and accesses the superclass's private data via inherited public methods. (Part 4 of 4.)



- ▶ earnings() metodu subclass içinde override edilmiştir.
- ▶ BasePlusCommissionEmployee nesne bu yeni versiyonu çağırır.
- ▶ Bu metod içerisinde ise superclass a ait (CommissionEmployee) earnings metodu çağırılır.
- ▶ **super . earnings()**
- ▶ Super anahtar kelimesi ile üst sınıftaki metotlar çağırılır.
- ▶ Bu gibi durumlarda kodu yeniden yazmak yerine metot çağırısı yapmak iyi bir mühendislik örneğidir.

COMMON PROGRAMMING ERROR



Bir alt sınıfta bir üst sınıf metodu override edildiğinde, alt sınıf sürümü genellikle işin bir kısmını yapması için üst sınıf sürümünü çağırır.

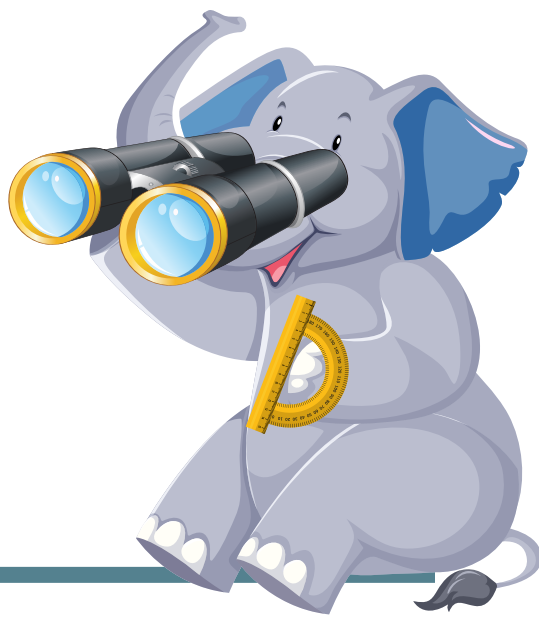
Üst sınıfın metodu çağrılırken üst sınıf metot adının önüne super anahtar kelimesi ve nokta (.) ayırıcısının konulmaması durumu alt sınıf metodunun kendisini çağırmasına neden olur.

Potansiyel olarak sonsuz özyineleme adı verilen ve sonunda yöntem çağrısı yığınının taşmasına neden olacak bir hata oluşturur.



- ▶ Bir alt sınıf nesnesinin somutlaştırılması, bir **CONSTRUCTOR** çağrılar zincirini başlatır.
- ▶ Alt sınıftaki constructor, kendi görevlerini gerçekleştirmeden önce,
 - ▶ Doğrudan üst sınıftaki constructor lardan birini çağırmak için açıkça **super** anahtar kelimesini kullanır.
 - ▶ Üst sınıfın varsayılan veya argümentsiz kurucusunu üstü kapalı olarak çağırır.
- ▶ Üst sınıf başka bir sınıftan türetilmişse, üst sınıf constructor hiyerarşide bir sonraki sınıfa ait constructor çağırır ve bu böyle devam eder.
- ▶ Zincirde çağrılan son constructor her zaman **Object** sınıfına ait constructor olacaktır.
- ▶ Orijinal alt sınıf constructor gövdesi en son yürütülür.
- ▶ Her bir üst sınıf constructor, alt sınıf nesnesinin devraldığı üst sınıf örnek değişkenlerini manipüle eder.

SOFTWARE ENGINEERING OBSERVATION



Java, bir constructor bir örnek değişkene bir değer atamasa bile, değişkenin yine de varsayılan değerine (örneğin, ilkel sayısal türler için 0, mantıksal değerler için false, referanslar için null) başlatılmasını sağlar.



- ▶ Tüm sınıflar doğrudan veya dolaylı olarak OBJECT sınıfını miras alırlar.
- ▶ Böylece bu sınıfa ait 11 metot miras alınmış olur.

1.equals

7.notify

2.hashCode

8.notifyAll

3.toString

9.getClass

4.wait()

10.finalize

5.wait(long timeout)

11.clone

6.wait(long timeout, int nanos)



1. **equals**: iki nesnenin eşitliğini kontrol eder.

- Default olarak `==` operatör kullanılır.
- Referanslar için bellekte aynı nesneye referans olma durumu kontrol edilir.

2. **hashCode**: Nesneye ait hashcode değerini getirir. (Hızlı bilgi getirimi)

3. **toString**: Nesne temsil eden String değeri getirir. (Paket adı, sınıf adı ve hashcode)

4. **wait**: ilgili thread durdurma işlemi yapar. (2 overload metot bulunur.)

7. **notify**: ilgili thread için uyandırma işlemi yapar.

8. **notifyAll**: Tüm threadler için uyandırma işlemi yapar.

9. **getClass**: Nesnenin sınıfını/tipini verir.

10. **finalize**: Garbage collection

11. **Clone**: Nesnenin kopyasını oluşturur.



- ▶ Yazılım mühendisliği topluluğunda, kompozisyon ve kalıtımın göreceli değerleri hakkında çok fazla tartışma var.
- ▶ Her birinin kendi yeri vardır, ancak kalıtım genellikle gereğinden fazla kullanılır ve birçok durumda kompozisyon daha uygundur.
- ▶ Alıştırma 9.1 6'da göreceğiniz gibi, kompozisyon ve kalıtımın bir karışımı genellikle makul bir tasarım yaklaşımıdır.

SOFTWARE ENGINEERING OBSERVATION



Bir üniversite öğrencisi olarak, çözüm üretmek için araçlar öğrenirsiniz. Endüstride problemler, üniversite derslerinde gördüğünüzden daha büyük ve daha karmaşıktır.

Genellikle çözmekte olduğunuz problemin talepleri benzersiz olacaktır ve elinizdeki araçları kullanmanın doğru yolunu yeniden düşünmenizi gerektirebilir.

Bir üniversite öğrencisi olarak, problemler üzerinde kendiniz çalışma eğilimindesiniz. Endüstride, problem çözme genellikle birçok meslektaş arasında etkileşim gerektirir. Nadiren bir projedeki herkesin bir çözüme "doğru" yaklaşım üzerinde anlaşmaya varmasını sağlayabilirsiniz.

Ayrıca, nadiren belirli bir yaklaşım "mükemmel" olacaktır. Bu bölümde yaptığımız gibi, farklı yaklaşımların görece avantajlarını sıklıkla karşılaştıracaksınız.



- ▶ Kalıtım, bir hiyerarşideki sınıflar arasında sıkı bağlantı oluşturur.
- ▶ Her alt sınıf tipik olarak doğrudan veya dolaylı üst sınıflarının uygulamalarına bağlıdır.
- ▶ Üst sınıf uygulamasındaki değişiklikler, genellikle ince yollarla alt sınıfların davranışını etkileyebilir.
- ▶ Sıkı bir şekilde birleştirilmiş tasarımların değiştirilmesi, gevşek bir şekilde birleştirilmiş, kompozisyona dayalı tasarımlara göre daha zordur.
- ▶ Değişim istisnadan ziyade bir kuraldır - bu, kompozisyonu teşvik eder :)
- ▶ Genel olarak, kalıtımı yalnızca bir üst sınıf referansına bir alt sınıf nesnesi atayabileceğiniz doğru “is-a” ilişkileri için kullanın.
- ▶ Bir alt sınıf nesnesine bir üst sınıf referansı aracılığıyla bir yöntem çağırdığınızda, alt sınıfın karşılık gelen yöntemi yürütülür.
- ▶ Buna, Bölüm 5’da inceleyeceğimiz “polimorfik” davranış denir.

SOFTWARE ENGINEERING OBSERVATION

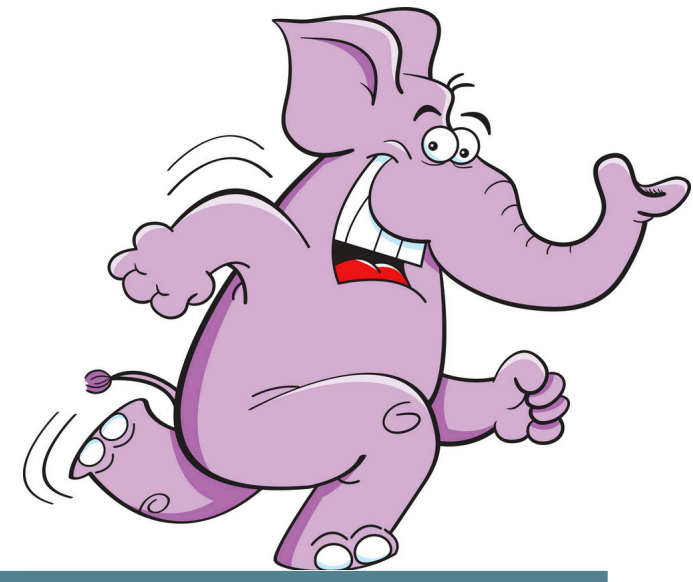


Büyük projelerde katılım hiyerarşisini farklı kişiler ile yönetmek zordur. Bu yüzden bu tasarım tek bir kişinin kontrolünde olmalıdır.



- ▶ Kompozisyon gevşek bir şekilde birleştirilmiştir.
- ▶ Bir sınıfın örnek değişkeni olarak bir referans oluşturduğunuzda, bu, sınıfın istemci kodundan gizlenen, sınıfın uygulama ayrıntılarının bir parçasıdır.
- ▶ Ayrıca kalıtım derleme zamanında yapılır.
- ▶ Kompozisyon daha esnektir - o da derleme zamanında yapılabilir, ancak oluşturulan nesnelere nihai olmayan referanslar değiştirilebildiği için yürütme zamanında da yapılabilir.(dinamik kompozisyon)
- ▶ Kalıtım yerine bir kompozisyon yaklaşımı kullandığınızda, genellikle her biri tek bir sorumluluğa odaklanan çok sayıda küçük sınıf oluşturursunuz.
- ▶ Daha küçük sınıfların test edilmesi, hatalarının ayıklanması ve değiştirilmesi genellikle daha kolaydır
- ▶ Java çoklu kalıtım sunmaz—Java'daki her sınıf yalnızca bir sınıfı genişletebilir.

PERFORMANCE TIP



Kompozisyonun olası bir dezavantajı, çalışma zamanında tipik olarak daha fazla nesne gerektirmesidir, bu da çöp toplama ve sanal bellek performansını olumsuz etkileyebilir.

Sanal bellek mimarileri ve performans sorunları genellikle işletim sistemleri derslerinde tartışılır.